



LxCameraSDK (Python)

开发指南

版本号: V2.0

发布日期: 2026.06.04

GitHub: <https://github.com/MRDVS/CameraSDK>

知识库: <https://hub.mrdvs.cn/>

官网: <https://www.mrdvs.cn/> & <https://mrdvs.com/>

浙江迈尔微视科技有限公司 版权所有



目录

一、概述.....	1
二、环境搭建.....	2
2.1 主机推荐系统配置.....	2
2.2 硬件环境搭建.....	2
2.3 网络环境搭建.....	3
2.4 软件环境搭建.....	4
三、SDK 目录与 Python 包配置.....	5
3.1 LxCameraSDK 目录结构.....	5
3.2 Python 包目录结构.....	5
3.3 安装与运行时库配置.....	6
3.4 最小工程配置.....	7
四、开发流程.....	7
4.1 接口调用总体原则.....	7
4.2 同步方式接口调用流程.....	8
4.3 状态回调开发流程.....	9
4.4 内置应用算法流程.....	10
五、接口调用注意事项.....	10
5.1 调用顺序与资源管理.....	10
5.2 部分接口调用前置条件.....	11
5.3 返回值处理.....	11
六、数据类型与枚举说明.....	12
6.1 LX_STATE.....	12
6.2 LX_DATA_TYPE.....	13
6.3 LX_DEVICE_TYPE.....	14
6.4 LX_DEVICE_SERIALS.....	15
6.5 LX_OPEN_MODE.....	15
6.6 LX_2D_OUTPUT_FORMAT.....	16
6.7 JPEG_DECODE_METHOD.....	16
6.8 LX_BINNING_MODE.....	16
6.9 LX_STRUCT_LIGHT_CODE_MODE.....	16
6.10 LX_ALGORITHM_MODE.....	17

6.11 LX_CAMERA_WORK_MODE	17
6.12 LX_TRIGGER_MODE	17
6.13 LX_IO_WORK_MODE	17
6.14 LX_IO_OUTPUT_STATE	18
6.15 LX_IO_HARDWARE_MODE	18
6.16 LX_FILTER_MODE	18
6.17 LX_3D_FREQ_MODE.....	18
6.18 LX_RGBD_ALIGN_MODE.....	19
6.19 LX_CAN_PROTOCOL_TYPE.....	19
6.20 LX_POWER_MODE_TYPE.....	19
6.21 LX_CAMERA_STATUS	19
6.22 LX_DISTORTION_MODEL	20
七、功能参数说明.....	20
7.1 int 型参数.....	20
7.2 float 型参数	25
7.3 bool 型参数.....	26
7.4 string 型参数.....	27
7.5 command 型参数	28
7.6 ptr 型参数	28
八、结构体说明.....	30
8.1 设备与参数信息结构体.....	30
8.2 帧数据与相机参数结构体.....	31
8.3 应用算法基础结构体.....	33
8.4 避障算法结构体.....	34
8.5 托盘与视觉定位结构体.....	36
九、API 说明	38
9.1 版本、日志和全局配置.....	38
9.2 查找、连接与关闭相机.....	38
9.3 开始取流与关闭取流.....	39
9.4 读取与设置相机参数.....	39
9.5 图像、点云和相机参数读取.....	40
9.6 特殊控制、ROI 和算法输出	40
9.7 状态回调.....	41

十、示例程序说明.....	41
10.1 示例程序索引.....	41
10.2 single_camera.py: 单相机同步取流.....	42
10.3 application_obstacle.py: 避障算法.....	42
10.4 application_pallet.py: 托盘对接算法.....	42
10.5 application_location.py: 视觉定位参考.....	43
10.6 utils.py: 错误处理与图像显示.....	43
十一、调试与常见问题.....	43
十二、版本与兼容说明.....	44

一、概述

本文面向使用 LxCameraSDK 接入 MRDVS/LxCamera 系列相机的 Python 开发人员，说明 Python 包安装、动态库加载、设备搜索与连接、参数配置、同步取流、状态回调、图像和点云数据读取，以及内置避障、托盘对接、视觉定位等应用算法的开发方法。

Python 接口通过 ctypes 加载底层 LxCameraApi 动态库，并把部分指针数据转换为 Python 对象或 numpy.ndarray。开发时既要关注 Python 包版本，也要关注实际加载的 LxCameraApi.dll 或 libLxCameraApi.so 版本。

表 1-1 SDK 能力范围

能力	说明
设备接入	支持搜索设备列表，并按索引、IP、SN 或 ID 打开设备。打开成功后返回 DCHandle，后续设备操作均依赖该句柄。
Python 封装	LxCamera 类封装底层 C 接口；枚举、结构体和回调类型位于 lx_camera_define.py 与 lx_camera_application.py。
参数配置	LX_CAMERA_FEATURE 按 LX_INT、LX_FLOAT、LX_BOOL、LX_STRING、LX_CMD、LX_PTR 前缀区分读写方式。Python 接口要求传入 LX_CAMERA_FEATURE 枚举对象。
图像取流	支持 2D/RGB、3D 深度、3D 强度流。同步方式通过 getFrame 触发 LX_CMD_GET_NEW_FRAME，再读取 FrameInfo 中的各类图像数据。
点云输出	getPointCloud 读取 LX_PTR_XYZ_DATA 并转换为 shape 为(height,width,3) 的 numpy.ndarray；DcSaveXYZ 支持 txt、ply、pcd 格式保存。
状态回调	支持注册相机状态回调，状态变化时返回 CameraStatus。当前 Python 包未封装帧回调和 IMU 回调函数。
内置算法	支持 MODE_AVOID_OBSTACLE、MODE_AVOID_OBSTACLE2、MODE_PALLET_LOCATE 和 MODE_VISION_LOCATION 等算法模式，算法结果通过 getAlgorithmStatus 读取。

表 1-2 当前版本信息

项目	内容
Python 包版本	lx-camera-py 1.3.3, whl 文件位于 Sample/python/lx_camera_py-1.3.3-py3-none-any.whl。
Python 封装 SDK_VERSION	2.4.30, 定义于 LxCameraSDK/lx_camera_api.py，用于运行时版本检查。
底层 API 版本宏	LX_CAMERA_API_VERSION V2.4.60,20260126, 定义于 SDK/include/lx_camera_api_version.h。

项目	内容
依赖包	numpy、opencv-python；示例中点云可视化额外使用 open3d，但不是 Python 包的必需依赖。
主要示例目录	Sample/python/lx_camera_py/LxCameraSDK/Sample。

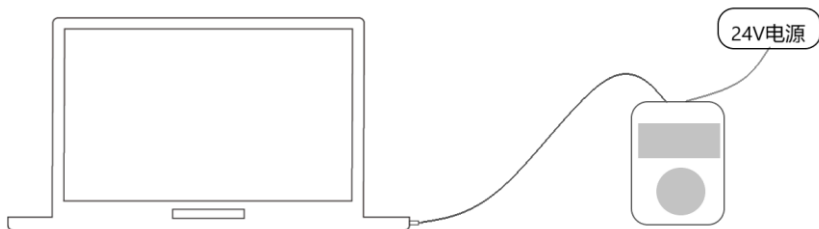
二、环境搭建

2.1 主机推荐系统配置

配置项	推荐配置	说明
操作系统	Windows7/10/11； Ubuntu16.04 及以上	Python 包可在 Windows 和 Linux 环境使用，动态库后缀分别为.dll 和.so。Linux Arm 环境需确认安装包与运行库架构匹配。
Python 版本	Python3.x	建议使用 64 位 Python 匹配 win_x64 动态库；若使用 32 位 Python，应加载 win_x86 动态库。
CPU	4 核及以上	多相机、点云转换、OpenCV 显示和应用算法输出建议使用更高主频或更多核心。
内存	4GB 及以上	RGBD、点云保存和多相机缓存会占用较多内存。
网卡	千兆及以上	深度、强度和 RGB 同时启流时建议使用独立千兆网口，避免与其他高带宽业务共用。
磁盘	保留日志和点云保存空间	pcd/ply/txt 点云保存可能快速占用磁盘，调试时注意清理。

2.2 硬件环境搭建

相机外接 24V 电源，通过网线连接到上位机所在主机，如图所示。RGBD、点云保存和多相机缓存会占用较多内存。



检查项	要求
供电	确认 24V 电源电压、电流和接线满足相机要求，避免供电不稳导致设备重启或断流。
网线	使用质量可靠的网线，确认主机网口协商到千兆；可通过 LX_INT_LINK_SPEED 获取协商网速。
IP 规划	默认相机网段通常为 192.168.100.*，主机网口需配置到同一网段且地址不冲突。
多机连接	多台相机避免重复 IP；TOF 多机互相干扰时，可结合 LX_BOOL_ENABLE_MULTI_MACHINE 和现场安装角度处理。

2.3 网络环境搭建

相机默认 IP 网段通常为 192.168.100.*。上位机与相机直连时，将对应网口 IPv4 地址设置为同一网段，例如 192.168.100.99，子网掩码 255.255.255.0。若相机 IP 已被修改，应以 DcGetDeviceList 返回的 LxDeviceInfo.ip 为准。



连接相机时建议关闭防火墙防止相机数据包被 SDK 所在主机拦截，如下图所示。



平台	建议操作
Windows	将网口 IPv4 地址配置到相机同一网段。调试阶段可临时关闭防火墙，或放行 Python 解释器和 SDK 示例程序的网络访问。
Ubuntu	网口配置同 Windows；防火墙可使用 <code>sudo systemctl stop ufw.service</code> 、 <code>sudo systemctl disable ufw.service</code> 、 <code>sudo ufw status</code> 检查。
跨网段	SDK 搜索依赖网络可达性。跨网段或多网卡环境下，优先确认路由、网关、子网掩码和相机实际 IP。
修改相机 IP	DcSetCameraIp 设置成功后相机会自动重启，设备列表会变化，必须重新调用 DcGetDeviceList 后再打开设备。

2.4 软件环境搭建

项目	说明
安装包	Sample/python/lx_camera_py-1.3.3-py3-none-any.whl。
源码包	Sample/python/lx_camera_py，包含 LxCameraSDK 包、示例和 dist-info 元数据。
必需依赖	numpy、opencv-python。opencv-python 主要用于示例图像显示和归一化伪彩色显示。
可选依赖	open3d 仅用于 single_camera.py 中的点云可视化示例，不影响 SDK 基础调用。
动态库	Windows 加载 LxCameraApi.dll；Linux 加载 libLxCameraApi.so。Python 包不自动搜索 SDK 目录，实例化 LxCamera 时需传入动态库路径。

Windows 示例

```
cd /d D:\Program Files\MRDVS\Sample\python
```

```
python -m pip install lx_camera_py-1.3.3-py3-none-any.whl
```

```
# Linux 示例
```

```
cd /opt/MRDVS/Sample/python
```

```
python3 -m pip install lx_camera_py-1.3.3-py3-none-any.whl
```

如果运行环境无法联网，应提前准备 numpy、opencv-python 和 open3d 等依赖的离线安装包。若只是调用 SDK 接口而不显示图像，可先安装 Python 包和 numpy，按实际需要再安装 OpenCV 相关依赖。

三、SDK 目录与 Python 包配置

3.1 LxCameraSDK 目录结构

目录	内容
Document	SDK 说明文档、开发指南、示例手册和常见问题。
FirmWare	相机固件升级包。
Sample	不同语言 and 不同功能场景的示例代码。Python 示例位于 Sample/python。
Sample/python	Python whl 安装包、源码包和打包脚本。
SDK/include	C/C++ 头文件。Python 封装中的枚举和结构体与这些头文件保持对应关系。
SDK/lib	Windows 平台动态库与链接库。Python 运行时需要加载对应架构的 LxCameraApi.dll。

3.2 Python 包目录结构

文件或目录	说明
LxCameraSDK/__init__.py	导出 lx_camera_define、lx_camera_api 和 lx_camera_application 中的主要对象，使用 from LxCameraSDK import * 时会加载这些定义。
LxCameraSDK/lx_camera_api.py	定义 LxCamera 类，负责加载动态库、初始化底层 C 函数签名，并提供 Python 调用方法。
LxCameraSDK/lx_camera_define.py	定义通用枚举、设备信息结构体、帧结构体、回调类型和 LX_CAMERA_FEATURE 参数枚举。
LxCameraSDK/lx_camera_application.py	定义避障、托盘、定位等应用算法相关结构体。

文件或目录	说明
LxCameraSDK/Sample/single_camera.py	单相机同步取流、状态回调、图像显示、点云读取和帧率读取示例。
LxCameraSDK/Sample/application_obstacle.py	避障算法模式设置、算法参数读取和算法输出读取示例。
LxCameraSDK/Sample/application_pallet.py	托盘对接算法模式设置和 LxPalletPose 结果读取示例。
LxCameraSDK/Sample/application_location.py	视觉定位相关结构参考。源码当前标注 TODO NOT FINISHED, DO NOT USE IT, 正式项目使用前需补全验证。
LxCameraSDK/Sample/utils.py	图像可视化和 LX_STATE 检查辅助函数。

3.3 安装与运行时库配置

Python 包安装完成后，代码中通过 LxCamera(dll_path)加载底层动态库。Windows 环境需保证 Python 架构与动态库架构一致；Linux 环境需保证 libLxCameraApi.so 及其依赖库可以被系统加载。

```
from LxCameraSDK import *

# Windows
camera = LxCamera(r"D:\Program Files\MRDVS\SDK\lib\win_x64\LxCameraApi.dll")

# Linux
# camera = LxCamera("/opt/MRDVS/lib/libLxCameraApi.so")
```

平台	动态库路径示例	注意事项
Windows x64	D:\Program Files\MRDVS\SDK\lib\win_x64\LxCameraApi.dll	使用 64 位 Python 时加载 win_x64 目录下的 DLL。必要时将 SDK/lib/win_x64 加入 PATH，或把相关 DLL 放到程序可访问目录。
Windows x86	D:\Program Files\MRDVS\SDK\lib\win_x86\LxCameraApi.dll	使用 32 位 Python 时加载 win_x86 目录下的 DLL。
Linux	/opt/MRDVS/lib/libLxCameraApi.so	执行 install.sh 后通常会设置 LD_LIBRARY_PATH。新终端需 source ~/.bashrc，非 root 用户需确认当前用户环境变量。

3.4 最小工程配置

最小 Python 工程不需要 CMake 或 Makefile，只需保证 Python 环境、LxCameraSDK 包和底层动态库路径正确。建议把相机 IP、动态库路径、日志路径等配置集中放在配置文件或脚本开头，方便现场调试。

文件	作用
main.py	设备搜索、打开、启流、取帧和关闭的主流程。
config.py 或 config.json	保存相机 IP、动态库路径、日志目录、启用数据流等配置。
utils.py	封装 check_state、图像显示、日志输出和资源释放逻辑。
requirements.txt	记录 numpy、opencv-python、open3d 等依赖。

四、开发流程

4.1 接口调用总体原则

Python 封装对 cmd 参数做了类型检查，应传入 LX_CAMERA_FEATURE 枚举对象，不建议直接传整数值。DcSetIntValue 支持传入普通 int，也支持传入 Enum 对象；如果 value 是 Enum，会自动取其.value。

Feature 前缀	接口	用途
LX_INT	DcSetIntValue / DcGetIntValue	读写整型参数，如曝光、图像尺寸、触发模式、算法模式、滤波等级等。
LX_FLOAT	DcSetFloatValue / DcGetFloatValue	读写浮点参数，如滤波强度、光强、帧率和温度等。
LX_BOOL	DcSetBoolValue / DcGetBoolValue	读写布尔开关，如数据流开关、自动曝光、反畸变、同步帧等。
LX_STRING	DcSetStringValue / DcGetStringValue	读写字符串参数，如固件名、算法参数、算法版本、参数导入导出路径等。
LX_CMD	DcSetCmd/getFrame	执行命令，如刷新帧、软触发、重启设备、恢复默认参数等。
LX_PTR	DcGetPtrValue	获取内部数据指针。常用场景已封装为 getFrame、getPointCloud、get2DIntricParam、get3DIntricParam、getAlgorithmStatus。

4.2 同步方式接口调用流程

步骤	接口/操作	说明
1	LxCamera(dll_path)	加载底层 LxCameraApi 动态库，初始化 C 函数签名，并检查运行时版本。
2	DcSetInfoOutput	按需要设置日志等级、是否屏幕输出、日志目录和语言。
3	DcGetDeviceList	搜索设备，返回 state、dev_list 和 dev_num。
4	DcOpenDevice	按 IP、SN、ID 或索引打开相机，返回 handle 和 LxDeviceInfo。
5	DcSetBoolValue	根据需要开启 2D、3D 深度、3D 强度流。
6	DcGetIntValue	读取图像宽高、通道数和数据类型。ROI、Binning 或 RGBD 对齐后需重新读取。
7	DcStartStream	开始取流。
8	getFrame	内部调用 LX_CMD_GET_NEW_FRAME，再读取 LX_PTR_FRAME_DATA。
9	getDepthImage/getAmplImage/getRGBImage	从 FrameInfo 转换对应 numpy.ndarray。
10	getPointCloud/DcSaveXYZ	读取 XYZ 点云或保存点云文件。
11	DcStopStream/DcCloseDevice	退出前停止取流并关闭设备。

```

from LxCameraSDK import *

camera = LxCamera(r"D:\Program Files\MRDVS\SDK\lib\win_x64\LxCameraApi.dll")
state, dev_list, dev_num = camera.DcGetDeviceList()
if state != LX_STATE.LX_SUCCESS or dev_num <= 0:
    raise RuntimeError("未搜索到相机")

state, handle, device_info = camera.DcOpenDevice(LX_OPEN_MODE.OPEN_BY_IP,
"192.168.100.82")
if state != LX_STATE.LX_SUCCESS:
    raise RuntimeError(camera.DcGetErrorString(state))

camera.DcSetBoolValue(handle, LX_CAMERA_FEATURE.LX_BOOL_ENABLE_3D_DEPTH_STREAM, True)
camera.DcSetBoolValue(handle, LX_CAMERA_FEATURE.LX_BOOL_ENABLE_3D_AMP_STREAM, True)
camera.DcSetBoolValue(handle, LX_CAMERA_FEATURE.LX_BOOL_ENABLE_2D_STREAM, True)
camera.DcStartStream(handle)

```

```

state, frame = camera.getFrame(handle)
if state == LX_STATE.LX_SUCCESS:
    state, depth = camera.getDepthImage(frame)
    state, amp = camera.getAmpImage(frame)
    state, rgb = camera.getRGBImage(frame)
camera.DcStopStream(handle)
camera.DcCloseDevice(handle)

```

4.3 状态回调开发流程

当前 Python 封装了相机状态回调，未封装 C/C++ 版本中的帧回调和 IMU 回调。状态回调适合监控相机打开、启流、断线重连等状态变化；图像采集仍按同步方式调用 `getFrame`。注意：当前 Python 封装在 `DcRegisterCameraStatusCallback` 内部创建 `ctypes` 回调对象，但未显式保存该对象引用。正式项目中如需长期使用状态回调，建议在封装层保存回调对象，避免回调对象被 Python 垃圾回收。

步骤	接口/操作	说明
1	定义 <code>callback(state_ptr)</code>	回调参数是 <code>CameraStatus</code> 指针，可通过 <code>state_ptr.contents</code> 读取 <code>camera_state</code> 和 <code>status_time</code> 。
2	<code>DcRegisterCameraStatusCallback</code>	打开设备后注册状态回调。
3	采集或等待状态变化	断线、重连、启流等状态变化时由 SDK 触发回调。
4	<code>DcUnregisterCameraStatusCallback</code>	关闭设备前注销回调，避免回调函数引用失效。

```

def camera_status_callback(status_ptr):
    status = status_ptr.contents
    print(status.camera_state, status.status_time)

state = camera.DcRegisterCameraStatusCallback(handle, camera_status_callback)

# 退出前
state = camera.DcUnregisterCameraStatusCallback(handle)

```

4.4 内置应用算法流程

步骤	接口/操作	说明
1	DcSetIntValue(LX_INT_ALGORITHM_MODE, mode)	选择 MODE_AVOID_OBSTACLE、MODE_AVOID_OBSTACLE2、MODE_PALLET_LOCATE 或 MODE_VISION_LOCATION。
2	DcGetStringValue(LX_STRING_ALGORITHM_VERSION)	读取当前算法模式对应版本。必须先设置算法模式。
3	DcGetStringValue / DcSetStringValue(LX_STRING_ALGORITHM_PARAMS)	读取或写入算法参数 JSON。写入前建议先 json.loads 检查格式，再 json.dumps 回写。
4	DcSetIntValue(LX_INT_WORK_MODE, WORK_FOREVER)	部分应用算法示例会设置常开模式，实际项目需根据相机型号和现场逻辑确认。
5	DcStartStream/getFrame	启流后循环取帧，确保算法输出数据刷新。
6	getAlgorithmStatus	按当前算法模式自动转换为 LxAvoidanceOutput、LxAvoidanceOutputN、LxPalletPose 或 LxLocation。
7	DcSpecialControl	特殊控制命令用于 GetObstacleIO、SetLaserData、ImportLocationMapFile 等扩展操作，命令和参数需按具体算法确认。

五、接口调用注意事项

5.1 调用顺序与资源管理

规则	说明
打开与关闭成对	DcOpenDevice 成功后必须调用 DcCloseDevice 释放资源和独占控制权限。程序异常退出未关闭时，需等待心跳超时后权限释放。
启流与停流成对	DcStartStream 成功后，退出前调用 DcStopStream；停流后再关闭设备。
回调先注销	使用状态回调时，关闭设备前应先调用 DcUnregisterCameraStatusCallback。
同步刷新	getFrame 内部会执行 LX_CMD_GET_NEW_FRAME；如果只读取点云，也需先调用 DcSetCmd 或 getFrame 刷新数据。
内部内存不要手动释放	DcGetDeviceList、DcGetStringValue、DcGetPtrValue 返回的内部内存通常由 SDK 管理。跨线程或长期保存时，应复制到 Python 对象或 numpy 数组。

规则	说明
动态库路径固定	LxCamera 初始化时不会自动搜索 SDK 安装目录，dll_path 必须指向当前平台可加载的动态库。

5.2 部分接口调用前置条件

类别	限制或前置条件
启流状态限制	启流状态或 LX_INT_WORK_MODE 为 WORK_FOREVER 常开模式时，不应设置 ROI、Binning、触发模式、计算上移、多机模式等需在启流前确定的参数。
自动曝光	3D 自动曝光开启时，不允许手动设置多曝光 HDR、曝光值和增益；2D 自动曝光开启时，不应设置 2D 手动曝光和手动增益。
图像尺寸	设置 ROI、Binning 或 RGBD 对齐后，3D/2D 图像宽高、通道数和数据类型可能变化，应重新调用 DcGetIntValue 获取。
点云读取	读取点云前必须开启 LX_BOOL_ENABLE_3D_DEPTH_STREAM，并保证已经通过 DcSetCmd 或 getFrame 刷新出有效帧。
算法参数	读写 LX_STRING_ALGORITHM_PARAMS 或 LX_STRING_ALGORITHM_VERSION 前必须先设置 LX_INT_ALGORITHM_MODE。
状态回调	回调函数对象需在注册期间保持有效，不要在局部作用域提前被释放。
ROI 接口	底层 C 接口名称为 DcSetROI。当前 Python 源码中 _init_c_func 配置的是 self._dll.DcSetROI，但封装方法 DcSetRoI 实际调用 self._dll.DcSetRoI，大小写不一致。正式使用 ROI 前应先修正 Python 封装，使方法调用 self._dll.DcSetROI，或自行扩展对应 ctypes 调用。

5.3 返回值处理

大部分 LxCamera 方法返回值第一个元素都是 LX_STATE。LX_STATE.LX_SUCCESS 表示成功，其余状态可通过 DcGetErrorString 查询底层错误说明。Python 项目中建议封装统一的 check_state 函数，避免每个调用点重复写错误判断。

```
def check_state(camera, state, handle=None):

    if state != LX_STATE.LX_SUCCESS:

        if state == LX_STATE.LX_E_RECONNECTING:

            print("device reconnecting")

            return

        print(camera.DcGetErrorString(state))
```

```

if handle is not None:

    camera.DcCloseDevice(handle)

    raise RuntimeError(state)

```

六、数据类型与枚举说明

本章数据类型、枚举和功能参数主要与 LxCameraSDK 底层 C/C++ 接口保持一致。Python 包通过 ctypes 封装底层动态库，当前 lx_camera_define.py 只显式定义了常用枚举、结构体和参数项，因此表中部分底层 SDK 支持的枚举或参数可能未在当前 Python 包中声明。若使用当前 Python 封装未声明的参数，不能直接向 DcSetIntValue、DcGetIntValue 等接口传入裸整数，因为封装代码会检查 cmd 是否为 LX_CAMERA_FEATURE 枚举对象；应先在 Python 封装中补充对应枚举，或按底层 C API 自行扩展 ctypes 调用。

6.1 LX_STATE

SDK 接口函数和帧状态使用的状态码。

关键字	值	说明
LX_SUCCESS	0	成功
LX_ERROR	-1	未知错误
LX_E_NOT_SUPPORT	-2	功能不支持
LX_E_NETWORK_ERROR	-3	网络通讯错误
LX_E_INPUT_ILLEGAL	-4	入参非法
LX_E_RECONNECTING	-5	设备重连中
LX_E_DEVICE_ERROR	-6	设备故障或设备响应失败
LX_E_DEVICE_NEED_UPDATE	-7	设备版本过低，需升级
LX_E_API_NEED_UPDATE	-8	API 版本过低
LX_E_CTRL_PERMISS_ERROR	-9	独占控制权限失败
LX_E_GET_DEVICEINFO_ERROR	-10	获取设备信息失败
LX_E_IMAGE_SIZE_ERROR	-11	图像尺寸不匹配，需重新打开
LX_E_IMAGE_PARTITION_ERROR	-12	图像解析失败（pixformat 不正确），需尝试重新打开
LX_E_DEVICE_NOT_CONNECTED	-13	相机未连接
LX_E_DEVICE_INIT_FAILED	-14	相机初始化失败

关键字	值	说明
LX_E_DEVICE_NOT_FOUND	-15	未找到相机（或未找到匹配的相机）
LX_E_FILE_INVALID	-16	文件错误（文件名或类型或格式不正确或文件打开失败）
LX_E_CRC_CHECK_FAILED	-17	文件 crc 或 md5 校验失败
LX_E_TIME_OUT	-18	超时
LX_E_FRAME_LOSS	-19	漏帧
LX_E_ENABLE_ANYSTREAM_FAILED	-20	开启任意流失败
LX_E_NOT_RECEIVE_STREAM	-21	未收到流数据
LX_E_PARSE_STREAM_FAILED	-22	启流成功但解析流数据失败
LX_E_PROCESS_IMAGE_FAILED	-23	图像计算处理失败
LX_E_SETTING_NOT_ALLOWED	-24	常开模式下不允许设置
LX_E_LOAD_DATAPROCESSLIB_ERROR	-25	加载图像处理算法库错误
LX_E_FUNCTION_CALL_LOGIC_ERROR	-26	函数调用逻辑错误
LX_E_IPAPPDR_UNREACHABLE_ERROR	-27	IP 不可达或网络配置错误
LX_E_FRAME_ID_NOT_MATCH	-28	超时范围内帧不同步错误
LX_E_FRAME_MULTI_MACHINE	-29	帧中检测到多机干扰信号
LX_E_FRAME_IMAGE_ERROR	-30	帧数据中检测到相机设备图像相关异常错误
LX_W_NOT_SUPPORT	2	功能不支持
LX_W_LOAD_DATAPROCESSLIB_ERROR	25	加载图像处理算法库失败，不影响其他功能

6.2 LX_DATA_TYPE

图像、点云和算法输出的数据格式。

关键字	值	说明
LX_DATA_UNSIGNED_CHAR	0	无符号字符型数据
LX_DATA_CHAR	1	有符号字符型数据
LX_DATA_UNSIGNED_SHORT	2	无符号短整型数据
LX_DATA_SIGNED_SHORT	3	有符号短整型数据

关键字	值	说明
LX_DATA_INT	4	整型数据
LX_DATA_FLOAT	5	浮点型数据
LX_DATA_DOUBLE	6	双精度浮点型数据
LX_DATA_OBSTACLE	16	避障算法结构体类型
LX_DATA_PALLET	17	托盘对接算法结构体类型
LX_DATA_LOCATION	18	视觉定位算法结构体类型
LX_DATA_OBSTACLE2	19	避障算法 V2 结构体类型
LX_DATA_CUSTOM	100	相机应用自定义的数据类型

6.3 LX_DEVICE_TYPE

设备型号枚举。实际支持能力以具体设备和固件为准。

关键字	值	说明
LX_DEVICE_M2	1001	M2 系列相机
LX_DEVICE_M3	1002	M3 系列相机
LX_DEVICE_M4Pro	1003	M4 Pro 系列相机
LX_DEVICE_M4_MEGA	1004	M4 MEGA 系列相机
LX_DEVICE_M4	1005	M4 系列相机
LX_DEVICE_M4V1_1	1006	M4 V1.1 系列相机
LX_DEVICE_M4ProV1_1	1007	M4 Pro V1.1 系列相机
LX_DEVICE_S1	2001	S1 系列相机
LX_DEVICE_S2	2002	S2 系列相机
LX_DEVICE_S2MaxV1	2003	S2 Max V1 系列相机
LX_DEVICE_S2MaxV2	2004	S2 Max V2 系列相机
LX_DEVICE_S2MaxV1_1	2005	S2 Max V1.1 系列相机
LX_DEVICE_S3	2101	S3 系列相机
LX_DEVICE_S10	2108	S10 系列相机
LX_DEVICE_S10PRO	2109	S10 PRO 系列相机
LX_DEVICE_S10ULTRA	2110	S10 ULTRA 系列相机

关键字	值	说明
LX_DEVICE_S11	2201	S11 系列相机
LX_DEVICE_I1	3001	I1 系列相机
LX_DEVICE_I2	3002	I2 系列相机
LX_DEVICE_T1	4001	T1 系列相机
LX_DEVICE_T2	4002	T2 系列相机
LX_DEVICE_H3	5001	H3 系列相机
LX_DEVICE_H4	5002	H4 系列相机
LX_DEVICE_V1Pro	6001	V1 Pro 系列相机
LX_DEVICE_V2Pro	6002	V2 Pro 系列相机
LX_DEVICE_NULL	0	预留或无效设备类型

6.4 LX_DEVICE_SERIALS

设备大类搜索过滤枚举。

关键字	值	说明
LX_SERIAL_GIGE	1	GigE 网络相机设备大类
LX_SERIAL_WK	2	WK 设备大类
LX_SERIAL_OTHER	3	其他设备大类
LX_SERIAL_ALL	4	搜索全部设备大类
LX_SERIAL_LOCAL_LOOP	5	本地回环（用于区分 SDK 与相机端程序运行在同一主机上的情况）

6.5 LX_OPEN_MODE

打开设备时 param 参数含义由打开方式决定。

关键字	值	说明
OPEN_BY_INDEX	0	按搜索列表中索引下标方式打开，对应的参数为索引号，当搜索到的设备列表发生变化时，选择打开的设备也会不一样
OPEN_BY_IP	1	按搜索列表中对应 ip 方式打开，对应的参数为设备 ip 或 ip:port
OPEN_BY_SN	2	按搜索列表中对应 sn 方式打开，对应的参数为设备 sn

关键字	值	说明
OPEN_BY_ID	3	按搜索列表中对 id 方式打开，对应的参数为设备 id

6.6 LX_2D_OUTPUT_FORMAT

2D 图像输出格式。

关键字	值	说明
LX_2D_FORMAT_RAW	0	RAW 原始 2D 图像格式
LX_2D_FORMAT_JPEG	4	此时，将不会进行后续的计算处理，包括去畸变、对齐等操作
LX_2D_FORMAT_YUV_NV12	5	YUV420
LX_2D_FORMAT_YUV_UYVY	6	YUV422
LX_2D_FORMAT_RGB	8	RGB888

6.7 JPEG_DECODE_METHOD

2D JPEG 图像解码方式。

关键字	值	说明
JPEG_DECODE_TURBOJPEG	0	turbojpeg
JPEG_DECODE_DP	1	使用 LxDataProcess
JPEG_DECODE_RKMPP	2	RK 硬解码，动态加载 librockchip_mpp.so
JPEG_DECODE_GPU	3	暂未实现

6.8 LX_BINNING_MODE

图像像素合并模式。

关键字	值	说明
LX_BINNING_1X1	0	1x1Binning，不进行像素合并
LX_BINNING_2X2	1	2x2Binning，2x2 像素合并
LX_BINNING_4X4	2	4x4Binning，4x4 像素合并

6.9 LX_STRUCT_LIGHT_CODE_MODE

结构光相机编码方式。

关键字	值	说明
LX_CODE_NORMAL	1	常规
LX_CODE_STATBLE	2	稳定
LX_CODE_ENHANCE	3	高精度加强

6.10 LX_ALGORITHM_MODE

相机内置应用算法模式。

关键字	值	说明
MODE_ALL_OFF	0	关闭内置避障算法
MODE_AVOID_OBSTACLE	1	内置避障算法
MODE_PALLET_LOCATE	2	内置托盘对接算法
MODE_VISION_LOCATION	3	内置视觉定位算法
MODE_AVOID_OBSTACLE2	4	内置避障算法 V2
MODE_GENERIC_DATA	5	自定义数据类型（应用算法平台）

6.11 LX_CAMERA_WORK_MODE

相机工作模式。

关键字	值	说明
KEEP_HEARTBEAT	0	保持心跳，SDK 心跳中断后相机自动待机
WORK_FOREVER	1	始终保持工作，此时部分参数无法设置

6.12 LX_TRIGGER_MODE

触发模式。

关键字	值	说明
LX_TRIGGER_MODE_OFF	0	关闭触发模式，流模式，默认
LX_TRIGGER_SOFTWARE	1	软触发模式
LX_TRIGGER_HARDWARE	2	硬触发模式

6.13 LX_IO_WORK_MODE

IO 工作模式。

关键字	值	说明
ALGORITHM_IO_MODE	0	IO 作为内置应用算法的输入输出
USER_IO_MODE	1	用户可以自己控制 IO 状态，参考 LX_IO_OUT_USERCTRL_MODE
TRIGGER_IO_MODE	2	IO 作为触发信号输出，仅触发模式下有效

6.14 LX_IO_OUTPUT_STATE

用户控制 IO 输出组合。

关键字	值	说明
OUT1_0_OUT2_0	0	OUT1 输出 0，OUT2 输出 0
OUT1_0_OUT2_1	1	OUT1 输出 0，OUT2 输出 1
OUT1_1_OUT2_0	2	OUT1 输出 1，OUT2 输出 0
OUT1_1_OUT2_1	3	OUT1 输出 1，OUT2 输出 1

6.15 LX_IO_HARDWARE_MODE

IO 输出硬件工作方式。

关键字	值	说明
OPEN_DRAIN_MODE	0	开漏常开模式
PUSH_PULL_MODE	1	推挽高电平模式

6.16 LX_FILTER_MODE

滤波参数模式。

关键字	值	说明
FILTER_SIMPLE	1	仅一个参数：LX_FLOAT_FILTER_LEVEL
FILTER_NORMAL	2	分多个模块：平滑，降噪，填充，时域
FILTER_EXPERT	3	通过字符串配置详细的算法

6.17 LX_3D_FREQ_MODE

3D 频率模式。

关键字	值	说明
FREQ_SINGLE_3D	0	单频

关键字	值	说明
FREQ_MULTI_3D	1	多频
FREQ_TRIPLE_3D	2	三频

6.18 LX_RGBD_ALIGN_MODE

RGBD 对齐模式。

关键字	值	说明
RGBD_ALIGN_OFF	0	关闭 rgb 对齐，rgb 和 depth 分别以各自分别和坐标系
DEPTH_TO_RGB	1	depth 对齐 rgb，分辨率和坐标以 rgb 为准，rgb 强制反畸变
RGB_TO_DEPTH	2	rgb 对齐 depth，分辨率和坐标以 depth 为准
DEPTH_TO_RGB_SPARSE	3	算力优化版 DEPTH_TO_RGB，分辨率和坐标以 rgb 为准，但不改变有效点云数据量

6.19 LX_CAN_PROTOCOL_TYPE

CAN 协议类型。

关键字	值	说明
CAN_MRDVS	0	MRDVS CAN 协议
CAN_KECONG	1	科聪 CAN 协议

6.20 LX_POWER_MODE_TYPE

相机功耗模式。

关键字	值	说明
POWER_MODE_LOW	0	低功耗模式（关闭部分功能）
POWER_MODE_DEFAULT	1	默认功耗模式（平衡性能与功耗）
POWER_MODE_HIGH	2	高性能模式（启用所有功能）
POWER_MODE_STANDBY	3	待机模式（最低功耗，暂停图像采集）

6.21 LX_CAMERA_STATUS

相机状态回调中的状态枚举。

关键字	值	说明
STATUS_CLOSED	0	相机未打开或关闭状态
STATUS_OPENED_UNSTARTED	1	相机已打开但未启流状态
STATUS_STARTED	2	相机打开且已启流状态
STATUS_CONNECTING	3	相机断线正在重连状态
STATUS_CONNECT_SUCCESS	4	相机重连成功

6.22 LX_DISTORTION_MODEL

相机内参结构中的畸变模型类型。

关键字	值	说明
LX_DISTORTION_UNDEFINE	0	无效畸变模型
LX_DISTORTION_RADTAN_5	5	常规径向切向模型 [k1, k2, p1, p2, k3]
LX_DISTORTION_RADTAN_14	14	大畸变径向切向模型 [k1, k2, p1, p2, k3, k4, k5, k6, s1, s2, s3, s4, t1, t2]
LX_DISTORTION_FISHEYE	15	鱼眼畸变模型 [k1, k2, k3, k4]
LX_DISTORTION_SCARAMUZZA	16	Scaramuzza 畸变模型

七、功能参数说明

下表“访问接口”沿用底层 LxCameraSDK 接口命名。Python 开发时通过 LxCamera 对象调用对应方法，例如 DcSetIntValue 在 Python 中写作 camera.DcSetIntValue(handle, LX_CAMERA_FEATURE.xxx, value)。对于 LX_PTR 类参数，当前 Python 包未公开 DcGetPtrValue 方法，常用指针读取已封装为 getFrame、getPointCloud、get2DIntricParam、get3DIntricParam、get3DTransMatrix 和 getAlgorithmStatus 等方法。

7.1 int 型参数

参数	编号	访问接口	说明
LX_INT_FIRST_EXPOSURE	1001	DcSetIntValue / DcGetIntValue	默认高积分曝光值，单位 us，针对多积分情况为第一个积分的曝光时间
LX_INT_SECOND_EXPOSURE	1002	DcSetIntValue / DcGetIntValue	默认低积分曝光值，单位 us，针对多积分情况为第二个积分的曝光时间
LX_INT_THIRD_EXPOSURE	1003	DcSetIntValue / DcGetIntValue	针对多积分情况为第三个积分的曝光时间，单位 us

参数	编号	访问接口	说明
LX_INT_FOURTH_EXPOSURE	1004	DcSetIntValue / DcGetIntValue	针对多积分情况为第四个积分的曝光时间，单位 us
LX_INT_GAIN	1005	DcSetIntValue / DcGetIntValue	增益，与曝光效果等价。会引入噪声，可适当调节增益防止曝光参数过大
LX_INT_MIN_DEPTH	1011	DcSetIntValue / DcGetIntValue	最小深度值
LX_INT_MAX_DEPTH	1012	DcSetIntValue / DcGetIntValue	最大深度值
LX_INT_MIN_AMPLITUDE	1013	DcSetIntValue / DcGetIntValue	有效信号最小强度值
LX_INT_MAX_AMPLITUDE	1014	DcSetIntValue / DcGetIntValue	有效信号最大强度值
LX_INT_CODE_MODE	1016	DcSetIntValue / DcGetIntValue	结构光相机编码模式，参考 LX_STRUCT_LIGHT_CODE_MODE
LX_INT_WORK_MODE	1018	DcSetIntValue / DcGetIntValue	工作模式，参考 LX_CAMERA_WORK_MODE
LX_INT_LINK_SPEED	1019	DcGetIntValue	协商的网卡网速 100-百兆，1000-千兆，只支持获取，不可设置
LX_INT_3D_IMAGE_WIDTH	1021	DcGetIntValue	3D 图像分辨率宽度，（ROI/Binning/2D 对齐后会变化，需重新获取）
LX_INT_3D_IMAGE_HEIGHT	1022	DcGetIntValue	3D 图像分辨率高度，（ROI/Binning/2D 对齐后会变化，需重新获取）
LX_INT_3D_IMAGE_OFFSET_X	1023	DcSetIntValue / DcGetIntValue	ROI 水平偏移像素，ROI 设置中 WIDTH HEIGHT OFFSET_X OFFSET_Y 一起设置，设置参数请用 DcSetROI
LX_INT_3D_IMAGE_OFFSET_Y	1024	DcSetIntValue / DcGetIntValue	ROI 垂直偏移像素，ROI 设置中 WIDTH HEIGHT OFFSET_X OFFSET_Y 一起设置，设置参数请用 DcSetROI
LX_INT_3D_BINNING_MODE	1025	DcSetIntValue / DcGetIntValue	3D 图像 binning，参考 LX_BINNING_MODE
LX_INT_3D_DEPTH_DATA_TYPE	1026	DcGetIntValue	深度图像数据格式，只能获取，对应的值参考 LX_DATA_TYPE

参数	编号	访问接口	说明
LX_INT_3D_FREQ_MODE	1027	DcSetIntValue / DcGetIntValue	3D 频率模式，详见 LX_3D_FREQ_MODE 定义
LX_INT_3D_AMPLITUDE_CHANNEL	1031	DcSetIntValue / DcGetIntValue	3D 强度图像通道数，与深度图通道共用，单色为 1，彩色为 3
LX_INT_3D_AMPLITUDE_DATA_TYPE	1035	DcGetIntValue	强度图像数据格式，只能获取，对应的值参考 LX_DATA_TYPE
LX_INT_3D_AUTO_EXPOSURE_LEVEL	1036	DcSetIntValue / DcGetIntValue	3D 自动曝光开启时的曝光等级，期间不允许设置曝光值与增益
LX_INT_3D_AUTO_EXPOSURE_MAX	1037	DcSetIntValue / DcGetIntValue	3D 自动曝光上限值
LX_INT_3D_AUTO_EXPOSURE_MIN	1038	DcSetIntValue / DcGetIntValue	3D 自动曝光下限值
LX_INT_2D_IMAGE_WIDTH	1041	DcSetIntValue / DcGetIntValue	2D 图像分辨率宽度，（ROI 或 Binning 后会变化）
LX_INT_2D_IMAGE_HEIGHT	1042	DcSetIntValue / DcGetIntValue	2D 图像分辨率高度，（ROI 或 Binning 后会变化）
LX_INT_2D_IMAGE_OFFSET_X	1043	DcSetIntValue / DcGetIntValue	2D 图像 ROI 水平偏移像素，ROI 设置中 WIDTH HEIGHT OFFSET_X OFFSET_Y 一起设置，设置参数请用 DcSetROI
LX_INT_2D_IMAGE_OFFSET_Y	1044	DcSetIntValue / DcGetIntValue	2D 图像 ROI 垂直偏移像素，ROI 设置中 WIDTH HEIGHT OFFSET_X OFFSET_Y 一起设置，设置参数请用 DcSetROI
LX_INT_2D_BINNING_MODE	1045	DcSetIntValue / DcGetIntValue	2D 图像 binning，参考 LX_BINNING_MODE
LX_INT_2D_IMAGE_CHANNEL	1046	DcSetIntValue / DcGetIntValue	2D 图像通道数，单色和 jpeg 为 1，彩色为 3，YUV422 为 2
LX_INT_2D_IMAGE_DATA_TYPE	1047	DcGetIntValue	2D 图像数据格式，只能获取，对应的值参考 LX_DATA_TYPE
LX_INT_2D_IMAGE_OUTPUT_FORMAT	1048	DcSetIntValue / DcGetIntValue	2D 图像输出格式，参考 LX_2D_OUTPUT_FORMAT
LX_INT_2D_ENCODE_RATIO	1049	DcSetIntValue / DcGetIntValue	默认 2D 图像以 jpeg 压缩，设置压缩比例。
LX_INT_2D_MANUAL_EXPOSURE	1051	DcSetIntValue / DcGetIntValue	2D 手动曝光时的曝光值

参数	编号	访问接口	说明
LX_INT_2D_MANUAL_GAIN	1052	DcSetIntValue / DcGetIntValue	2D 手动曝光时的增益值
LX_INT_2D_AUTO_EXPOSURE_LEVEL	1054	DcSetIntValue / DcGetIntValue	2D 图像自动曝光时曝光等级[0-100], 0-整体更暗, 100-整体更亮
LX_INT_TOF_GLOBAL_OFFSET	1061	DcSetIntValue / DcGetIntValue	3D 深度数据偏移
LX_INT_3D_UNDISTORT_SCALE	1062	DcSetIntValue / DcGetIntValue	3D 图像反畸变系数, [0,100], 值越大, 保留视野越多, 0 为保持原内参, 1 为无黑边
LX_INT_ALGORITHM_MODE	1065	DcSetIntValue / DcGetIntValue	设置内置应用算法, 部分型号支持, 对应的值参考 LX_ALGORITHM_MODE
LX_INT_MODBUS_ADDR	1066	DcSetIntValue / DcGetIntValue	modbus 地址, 部分型号支持 MODBUS 协议通过串口输出
LX_INT_HEART_TIME	1067	DcSetIntValue / DcGetIntValue	与设备间心跳时间, 单位 ms
LX_INT_GVSP_PACKET_SIZE	1068	DcSetIntValue / DcGetIntValue	GVSP 单包数据分包大小, 单位字节
LX_INT_TRIGGER_MODE	1069	DcSetIntValue / DcGetIntValue	触发模式, 对应的值参考 LX_TRIGGER_MODE
LX_INT_CALCULATE_UP	1070	DcSetIntValue / DcGetIntValue	允许 2d 或 3d 算法上下移, 节省上位机或相机算力, 可能影响帧率和延时
LX_INT_CAN_BAUD_RATE	1072	DcSetIntValue / DcGetIntValue	can 的波特率值, 单位 bps
LX_INT_CAN_NODE_ID	1073	DcSetIntValue / DcGetIntValue	can 地址
LX_INT_CAN_PROTOCOL_TYPE	1074	DcSetIntValue / DcGetIntValue	can 协议类型, 参考枚举 LX_CAN_PROTOCOL_TYPE
LX_INT_SAVE_PARAMS_GROUP	1075	DcSetIntValue / DcGetIntValue	将相机当前配置保存为指定的参数组
LX_INT_LOAD_PARAMS_GROUP	1076	DcSetIntValue / DcGetIntValue	一键加载指定索引的参数组
LX_INT_RGBD_ALIGN_MODE	1077	DcSetIntValue / DcGetIntValue	RGBD 对齐方式, 参考 LX_RGBD_ALIGN_MODE。用于替代历史兼容项 LX_BOOL_ENABLE_2D_TO_DEPTH。

参数	编号	访问接口	说明
LX_INT_XYZ_COORDINATE	1078	DcSetIntValue / DcGetIntValue	点云坐标系，0：相机坐标系，xyz 为右下前。1：机器人坐标系，xyz 为前左上。
LX_INT_XYZ_UNIT	1079	DcSetIntValue / DcGetIntValue	点云单位，0：mm，1：m。仅影响 LX_PTR_XYZ_DATA 和 LX_PTR_XYZIRT_DATA 的数据
LX_INT_GVCP_TIME_OUT	1080	DcGetIntValue	设备控制，信息获取最大超时时间 默认 1800ms 单位 ms
LX_INT_HARDWARE_TRIGGER_FILTER_TIME	1085	DcSetIntValue / DcGetIntValue	硬触发滤波时间，单位 us
LX_INT_TRIGGER_MIN_PERIOD_TIME	1086	DcSetIntValue / DcGetIntValue	触发最小时间间隔，单位 us
LX_INT_TRIGGER_DELAY_TIME	1087	DcSetIntValue / DcGetIntValue	触发延迟时间，单位 us
LX_INT_TRIGGER_FRAME_COUNT	1088	DcSetIntValue / DcGetIntValue	单次触发帧数
LX_INT_FILTER_MODE	1090	DcSetIntValue / DcGetIntValue	滤波模式，参考 LX_FILTER_MODE
LX_INT_FILTER_SMOOTH_LEVEL	1091	DcSetIntValue / DcGetIntValue	当 LX_INT_FILTER_MODE 为 FILTER_NORMAL 时，可设置滤波平滑等级，[0, 3]，值越大，滤波越强
LX_INT_FILTER_NOISE_LEVEL	1092	DcSetIntValue / DcGetIntValue	当 LX_INT_FILTER_MODE 为 FILTER_NORMAL 时，可设置滤波噪声等级，[0, 3]，值越大，滤波越强
LX_INT_FILTER_TIME_LEVEL	1093	DcSetIntValue / DcGetIntValue	当 LX_INT_FILTER_MODE 为 FILTER_NORMAL 时，可设置滤波时域等级，[0, 3]，值越大，滤波越强
LX_INT_FILTER_DETECT_LOW_SIGNAL	1094	DcSetIntValue / DcGetIntValue	小信号测量，可能检测到信噪比较低的数据，包括远距离数据导致多周期问题
LX_INT_FILTER_FILL_LEVEL	1095	DcSetIntValue / DcGetIntValue	当 LX_INT_FILTER_MODE 为 FILTER_NORMAL 时，可设置滤波填充等级，[0, 3]，值越大，滤波越强
LX_INT_IMU_ACCELERATION_LEVEL	1101	DcSetIntValue / DcGetIntValue	IMU 加速度量程等级，[0-3]，值越大量程越大，每一级变化一倍
LX_INT_IMU_ANGULAR_RANGE_LEVEL	1102	DcSetIntValue / DcGetIntValue	IMU 角速度量程等级，[0-4]，值越大量程越大，每一级变化一倍

参数	编号	访问接口	说明
LX_INT_MAX_TOLERANT_LOST_PACKETS	1103	DcSetIntValue / DcGetIntValue	一帧数据中允许丢包的数量，超出阈值则丢弃该帧，否则丢的包用 0 填充，默认为 5
LX_INT_3D_FPS	1510	DcSetIntValue / DcGetIntValue	深度、强度的帧率
LX_INT_2D_UNDISTORT_SCALE	1520	DcSetIntValue / DcGetIntValue	2D 图像反畸变系数，[0,100]，值越大，保留视野越多，0 为保持原内参，1 为无黑边
LX_INT_IO_WORK_MODE	1530	DcSetIntValue / DcGetIntValue	GPIO 信号输出控制模式，参考 LX_IO_WORK_MODE
LX_INT_IO_OUTPUT_STATE	1531	DcSetIntValue / DcGetIntValue	GPIO 信号输出的用户控制模式，参考 LX_IO_OUTPUT_STATE
LX_INT_IO_HARDWARE_WORK_MODE	1532	DcSetIntValue / DcGetIntValue	GPIO 信号输出的硬件工作模式，参考 LX_IO_HARDWARE_MODE
LX_INT_IO_INPUT_STATUS	1533	DcGetIntValue	IO 输入状态，只读
LX_INT_IO_OUTPUT_STATUS	1534	DcGetIntValue	IO 输出状态，只读
LX_INT_3D_GLARE_LEVEL	1738	DcSetIntValue / DcGetIntValue	炫光等级，0 关闭，等级 1,2,3
LX_INT_POWER_MODE	1755	DcSetIntValue / DcGetIntValue	相机功耗模式，详见 LX_POWER_MODE_TYPE 定义，部分型号支持

7.2 float 型参数

参数	编号	访问接口	说明
LX_FLOAT_FILTER_LEVEL	2001	DcSetFloatValue / DcGetFloatValue	当 LX_INT_FILTER_MODE 为 FILTER_SIMPLE 时，可设置滤波等级，[0, 1]，值越大，滤波越强，等于 0 表示关闭滤波
LX_FLOAT_EST_OUT_EXPOSURE	2002	DcSetFloatValue / DcGetFloatValue	是否评估过曝数据，[0, 1]，为 1 则过曝数据无效
LX_FLOAT_LIGHT_INTENSITY	2003	DcSetFloatValue / DcGetFloatValue	光强度，[0, 1]，部分型号支持
LX_FLOAT_3D_DEPTH_FPS	2004	DcGetFloatValue	深度图当前帧率，只可获取

参数	编号	访问接口	说明
LX_FLOAT_3D_AMPLITUDE_FPS	2005	DcGetFloatValue	强度图当前帧率， 只可获取
LX_FLOAT_2D_IMAGE_FPS	2006	DcGetFloatValue	2D/RGB 图当前帧率， 只可获取
LX_FLOAT_DEVICE_TEMPERATURE	2007	DcGetFloatValue	相机当前温度， 只可获取
LX_FLOAT_CONFIDENCE	2008	DcSetFloatValue / DcGetFloatValue	置信度， [0-1]， 越小则保留越多数据

7.3 bool 型参数

参数	编号	访问接口	说明
LX_BOOL_CONNECT_STATE	3001	DcSetBoolValue / DcGetBoolValue	当前连接状态
LX_BOOL_ENABLE_3D_DEPTH_STREAM	3002	DcSetBoolValue / DcGetBoolValue	开启/关闭深度数据流（部分相机支持）
LX_BOOL_ENABLE_3D_AMP_STREAM	3003	DcSetBoolValue / DcGetBoolValue	开启/关闭强度数据流（部分相机支持）
LX_BOOL_ENABLE_3D_AUTO_EXPOSURE	3006	DcSetBoolValue / DcGetBoolValue	3D 自动曝光使能
LX_BOOL_ENABLE_3D_UNDISTORT	3007	DcSetBoolValue / DcGetBoolValue	3D 反畸变使能
LX_BOOL_ENABLE_ANTI_FLICKER	3008	DcSetBoolValue / DcGetBoolValue	抗频闪使能，LED 环境照明可能导致数据存在明显波纹，部分型号支持
LX_BOOL_ENABLE_2D_STREAM	3011	DcSetBoolValue / DcGetBoolValue	开启/关闭 2D 数据流
LX_BOOL_ENABLE_2D_AUTO_EXPOSURE	3012	DcSetBoolValue / DcGetBoolValue	2D 自动曝光使能
LX_BOOL_ENABLE_2D_UNDISTORT	3015	DcSetBoolValue / DcGetBoolValue	2D 图像反畸变使能
LX_BOOL_ENABLE_2D_TO_DEPTH	3016	DcSetBoolValue / DcGetBoolValue	建议使用 LX_INT_RGBD_ALIGN_MODE
LX_BOOL_ENABLE_BACKGROUND_AMP	3017	DcSetBoolValue / DcGetBoolValue	强度背景光使能
LX_BOOL_ENABLE_MULTI_MACHINE	3018	DcSetBoolValue / DcGetBoolValue	多机模式使能，TOF 会有多机干扰，开启此模式可以自动检测并缓解多机干扰

参数	编号	访问接口	说明
LX_BOOL_ENABLE_MULTI_EXPOSURE_HDR	3019	DcSetBoolValue / DcGetBoolValue	HDR（多曝光高动态范围模式）使能
LX_BOOL_ENABLE_SYNC_FRAME	3020	DcSetBoolValue / DcGetBoolValue	是否开启强制帧同步，默认数据实时性优先，若需要 RGBD 同步，需要开启该模式
LX_BOOL_ENABLE_TERM_RESISTOR	3021	DcSetBoolValue / DcGetBoolValue	端接电阻使能
LX_BOOL_ENABLE_IMU	3023	DcSetBoolValue / DcGetBoolValue	使能 IMU 数据，通过回调 DcRegisterImuDataCallback 获取数据
LX_BOOL_ENABLE_LASER_BRIGHT	3110	DcSetBoolValue / DcGetBoolValue	激光器亮灭使能

7.4 string 型参数

参数	编号	访问接口	说明
LX_STRING_DEVICE_VERSION	4001	DcSetStringValue / DcGetStringValue	设备版本号
LX_STRING_FIRMWARE_NAME	4003	DcSetStringValue	固件文件名，用于升级设备版本，部分型号需要重新打开相机
LX_STRING_FILTER_PARAMS	4004	DcSetStringValue / DcGetStringValue	建议使用 FILTER_NORMAL
LX_STRING_ALGORITHM_PARAMS	4005	DcSetStringValue / DcGetStringValue	内置算法参数，根据当前设置的 LX_ALGORITHM_MODE，返回对应的 json 格式字符串
LX_STRING_ALGORITHM_VERSION	4006	DcSetStringValue / DcGetStringValue	内置算法版本号，根据当前设置的 LX_ALGORITHM_MODE，返回对应的版本号
LX_STRING_DEVICE_OS_VERSION	4007	DcSetStringValue / DcGetStringValue	设备系统镜像版本号
LX_STRING_IMPORT_PARAMS_FROM_FILE	4008	DcSetStringValue / DcGetStringValue	从本地文件加载参数到相机
LX_STRING_EXPORT_PARAMS_TO_FILE	4009	DcSetStringValue / DcGetStringValue	将相机当前参数导出到本地文件
LX_STRING_CUSTOM_ID	4010	DcSetStringValue / DcGetStringValue	客户自定义设备 ID 标识

7.5 command 型参数

参数	编号	访问接口	说明
LX_CMD_GET_NEW_FRAME	5001	DcSetCmd	主动更新当前最新数据。同步取流获取 LX_PTR_FRAME_DATA、LX_PTR_XYZ_DATA、LX_PTR_XYZIRT_DATA 或算法输出前调用；回调方式不需要调用。
LX_CMD_RETURN_VERSION	5002	DcSetCmd	回退上一版本
LX_CMD_RESTART_DEVICE	5003	DcSetCmd	重启相机，部分型号需要重新打开相机
LX_CMD_RESET_PARAM	5007	DcSetCmd	恢复默认参数
LX_CMD_SOFTWARE_TRIGGER	5008	DcSetCmd	软触发执行指令
LX_CMD_GET_NEW_FRAME_3D	5009	DcSetCmd	仅处理并更新 3D 数据 depth/amp，需要降低 2D 处理占用时使用。
LX_CMD_GET_NEW_FRAME_2D	5010	DcSetCmd	仅处理并更新 2D/RGB 数据，需要降低 3D 处理占用时使用。

7.6 ptr 型参数

参数	编号	访问接口	说明
LX_PTR_2D_IMAGE_DATA	6001	DcGetPtrValue	兼容接口，不建议新工程优先使用；推荐使用 LX_PTR_FRAME_DATA 中的 rgb_data。
LX_PTR_3D_AMP_DATA	6002	DcGetPtrValue	兼容接口，不建议新工程优先使用；推荐使用 LX_PTR_FRAME_DATA 中的 amp_data。
LX_PTR_3D_DEPTH_DATA	6003	DcGetPtrValue	兼容接口，不建议新工程优先使用；推荐使用 LX_PTR_FRAME_DATA 中的 depth_data。
LX_PTR_XYZ_DATA	6004	DcGetPtrValue	获取普通 XYZ 点云，float 三通道按 x、y、z 顺序排列；需开启深度流并完成取帧。

参数	编号	访问接口	说明
LX_PTR_2D_INTRIC_PARAM	6005	DcGetPtrValue	兼容接口，推荐使用 LX_PTR_2D_INTRINSIC_PARAMETERS。
LX_PTR_3D_INTRIC_PARAM	6006	DcGetPtrValue	兼容接口，推荐使用 LX_PTR_3D_INTRINSIC_PARAMETERS。
LX_PTR_3D_EXTRIC_PARAM	6007	DcGetPtrValue	获取 3D 图像外参，float*类型指针，长度固定为 12*sizeof(float)（前 9 个表示旋转矩阵，后 3 个表示平移向量）
LX_PTR_ALGORITHM_OUTPUT	6008	DcGetPtrValue	兼容接口，用于直接读取当前算法输出；示例中的避障 V2、托盘、定位程序仍可使用。通用帧回调场景优先使用 FrameInfo.app_data。
LX_PTR_FRAME_DATA	6009	DcGetPtrValue	获取完整一帧数据，输出 FrameInfo，推荐用于深度、强度、RGB 和算法输出的统一读取。
LX_PTR_2D_NEW_INTRIC_PARAM	6010	DcGetPtrValue	建议使用 LX_PTR_2D_INTRINSIC_PARAMETERS,float*类型指针，长度固定为 18（4 内参 14 畸变）
LX_PTR_3D_NEW_INTRIC_PARAM	6011	DcGetPtrValue	建议使用 LX_PTR_3D_INTRINSIC_PARAMETERS,float*类型指针，长度固定为 18（4 内参 14 畸变）
LX_PTR_2D_INTRINSIC_PARAMETERS	6013	DcGetPtrValue	获取 2D 图像内参，参考 LxIntrinsicParameters
LX_PTR_3D_INTRINSIC_PARAMETERS	6014	DcGetPtrValue	获取 3D 图像内参，参考 LxIntrinsicParameters
LX_PTR_IMU_EXTRIC_PARAM	6015	DcGetPtrValue	获取 IMU 图像外参，float*类型指针，长度固定为 12*sizeof(float)（前 9 个表示旋转矩阵，后 3 个表示平移向量）
LX_PTR_XYZIRT_DATA	6020	DcGetPtrValue	获取雷达点云结构 LxPointCloudData，包含 timebase、point_num 和 LxPointXYZIRT 点数组

八、结构体说明

8.1 设备与参数信息结构体

DCHandle

字段	ctypes 类型	说明
handle	c_ulonglong	设备句柄，打开设备成功后用于后续所有设备操作。

LxDeviceInfo

字段	ctypes 类型	说明
handle	DCHandle	设备唯一标识，打开设备后续接口依赖此句柄
dev_type	c_long	设备类型
id	c_char_Array_32	设备 ID
ip	c_char_Array_32	设备 IP 和端口
sn	c_char_Array_32	设备序列号
mac	c_char_Array_32	设备 MAC 地址
firmware_ver	c_char_Array_32	设备软件版本号
algor_ver	c_char_Array_32	设备算法版本号
name	c_char_Array_32	设备名称
reserve	c_char_Array_32	预留字段，当前用于子网掩码
reserve2	c_char_Array_32	预留字段，当前用于网关 IP
reserve3	c_char_Array_64	预留字段
reserve4	c_char_Array_128	预留字段

LxIntValueInfo

字段	ctypes 类型	说明
set_available	c_bool	当前值是否可设置。true 表示可设置，false 表示不可设置
cur_value	c_int	当前值
max_value	c_int	最大值
min_value	c_int	最小值

字段	ctypes 类型	说明
reserve	c_int_Array_4	预留字段

LxFloatValueInfo

字段	ctypes 类型	说明
set_available	c_bool	当前值是否可设置。true 表示可设置，false 表示不可设置
cur_value	c_float	当前值
max_value	c_float	最大值
min_value	c_float	最小值
reserve	c_float_Array_4	预留字段

8.2 帧数据与相机参数结构体**FrameDataInfo**

字段	ctypes 类型	说明
frame_data_type	c_long	帧数据类型
frame_width	c_long	图像宽度
frame_height	c_long	图像高度
frame_channel	c_long	通道数
frame_data	c_void_p	图像数据指针，Python 封装会转换为 numpy.ndarray。
sensor_timestamp	c_ulonglong	传感器出图时间戳
recv_timestamp	c_ulonglong	SDK 接收帧数据时间戳

FrameInfo

字段	ctypes 类型	说明
frame_state	c_long	当前帧状态。应先判断是否为 LX_SUCCESS
handle	DcHandle	设备句柄
depth_data	FrameDataInfo	深度图数据
amp_data	FrameDataInfo	强度图数据
rgb_data	FrameDataInfo	2D/RGB 图数据

字段	ctypes 类型	说明
app_data	FrameDataInfo	算法输出数据，类型由当前算法模式决定
reserve_data	c_void_p	扩展预留字段。当前 Python 示例未直接解析该字段；如需获取扩展帧 ID，可参考底层结构定义自行转换并验证。

FrameExtendInfo

字段	ctypes 类型	说明
depth_frame_id	c_ulong	深度图帧 ID
amp_frame_id	c_ulong	强度图帧 ID
rgb_frame_id	c_ulong	RGB 图帧 ID
app_frame_id	c_ulong	应用算法输出帧 ID
reserve_data1/2/3	c_void_p	保留指针

ImageIntricParam

字段	ctypes 类型	说明
fx	c_float	焦距参数 fx。
fy	c_float	焦距参数 fy。
cx	c_float	主点 cx。
cy	c_float	主点 cy。
d1	c_float	畸变参数 1。
d2	c_float	畸变参数 2。
d3	c_float	畸变参数 3。
d4	c_float	畸变参数 4。
d5	c_float	畸变参数 5。

Image3DExtricParam

字段	ctypes 类型	说明
r0	c_float	旋转矩阵第 1 行第 1 列。
r1	c_float	旋转矩阵第 1 行第 2 列。
r2	c_float	旋转矩阵第 1 行第 3 列。

字段	ctypes 类型	说明
r3	c_float	旋转矩阵第 2 行第 1 列。
r4	c_float	旋转矩阵第 2 行第 2 列。
r5	c_float	旋转矩阵第 2 行第 3 列。
r6	c_float	旋转矩阵第 3 行第 1 列。
r7	c_float	旋转矩阵第 3 行第 2 列。
r8	c_float	旋转矩阵第 3 行第 3 列。
t0	c_float	平移向量 x。
t1	c_float	平移向量 y。
t2	c_float	平移向量 z。

CameraStatus

字段	ctypes 类型	说明
camera_state	c_long	相机状态
status_time	c_ulonglong	状态变化时间
reserve_data	c_void_p	扩展预留字段

8.3 应用算法基础结构体**LxGroundPlane**

字段	ctypes 类型	说明
a	c_float	地面平面参数 a。
b	c_float	地面平面参数 b。
c	c_float	地面平面参数 c。
d	c_float	地面平面参数 d。

LxPoint2d

字段	ctypes 类型	说明
x	c_float	二维点 x 坐标。
y	c_float	二维点 y 坐标。

LxPoint3dWithRGB

字段	ctypes 类型	说明
x	c_float	三维点 x 坐标。
y	c_float	三维点 y 坐标。
z	c_float	三维点 z 坐标。
r	c_ubyte	红色分量。
g	c_ubyte	绿色分量。
b	c_ubyte	蓝色分量。

LxVelocity

字段	ctypes 类型	说明
linear	c_float_Array_3	线速度，长度为 3。
angular	c_float_Array_3	角速度，长度为 3。

LxPose 字段说明

字段	ctypes 类型	说明
R	c_float_Array_9	旋转矩阵，长度为 9。
T	c_float_Array_3	平移向量，长度为 3。
timestamp	c_ulonglong	位姿时间戳。

8.4 避障算法结构体**LxObstacleBox**

字段	ctypes 类型	说明
pose	LxPose	障碍物位姿。
width	c_float	障碍物宽度。
height	c_float	障碍物高度。
depth	c_float	障碍物深度。
center	c_float_Array_3	障碍物中心点。
type	c_long	障碍物类型。
id	c_ulonglong	障碍物 ID。
velocity	LxVelocity	障碍物速度。

LxObstacleBoxN

字段	ctypes 类型	说明
pose	LxPose	障碍物位姿。
width	c_float	障碍物宽度。
height	c_float	障碍物高度。
depth	c_float	障碍物深度。
center	c_float_Array_3	障碍物中心点。
type_idx	c_long	障碍物类型索引。
id	c_longlong	当前障碍物 ID。
prev_id	c_longlong	上一帧关联 ID。
box_2d_x_min	c_float	2D 检测框 x 最小值。
box_2d_y_min	c_float	2D 检测框 y 最小值。
box_2d_x_max	c_float	2D 检测框 x 最大值。
box_2d_y_max	c_float	2D 检测框 y 最大值。
velocity	LxVelocity	障碍物速度。
type_name_len	c_long	类型名称长度。
type_name	c_char_Array_512	类型名称。
reserved	c_char_Array_1024	保留字段。

LxAvoidanceOutput

字段	ctypes 类型	说明
state	c_long	避障算法状态。
groundPlane	LxGroundPlane	地面平面参数。
number_3d	c_ulong	输出点数量。
cloud_output	LP_LxPoint3dWithRGB	带 RGB 的三维点云指针。
number_box	c_ulong	障碍物框数量。
obstacleBoxes	LP_LxObstacleBox	障碍物框数组指针。

LxAvoidanceOutputN

字段	ctypes 类型	说明
state	c_int	避障 V2 算法状态。
groundPlane	LxGroundPlane	地面平面参数。
number_3d	c_uint32	输出点数量。
cloud_output	POINTER(LxPoint3dWithRGB)	带 RGB 的三维点云指针。
number_box	c_uint32	障碍物框数量。
obstacleBoxs	POINTER(LxObstacleBoxN)	避障 V2 障碍物框数组指针
resveredData	c_char_Array_4096	保留字段，源码字段名为 resveredData

8.5 托盘与视觉定位结构体

LxPalletPose

字段	ctypes 类型	说明
return_val	c_long	托盘算法返回值。
x	c_float	托盘位姿 x。
y	c_float	托盘位姿 y。
yaw	c_float	托盘航向角。
extents	c_long_Array_8192	扩展字段。

LxRelocPose

字段	ctypes 类型	说明
x	c_double	重定位位姿 x。
y	c_double	重定位位姿 y。
theta	c_double	重定位角度。

LxOdomData

字段	ctypes 类型	说明
timestamp	c_longlong	里程计时间戳。
x	c_double	里程计 x。
y	c_double	里程计 y。

字段	ctypes 类型	说明
theta	c_double	里程计角度。

LxLaser

字段	ctypes 类型	说明
timestamp	c_longlong	激光数据时间戳。
time_increment	c_float	扫描点时间间隔。
angle_min	c_float	最小角度。
angle_max	c_float	最大角度。
angle_increment	c_float	角度增量。
range_min	c_float	最小量程。
range_max	c_float	最大量程。
reserved	c_float	保留字段。
range_size	c_float	距离数组长度。
ranges	LP_c_float	距离数组指针。

LxLaserPose

字段	ctypes 类型	说明
timestamp	c_longlong	激光位姿时间戳。
x	c_double	激光位姿 x。
y	c_double	激光位姿 y。
theta	c_double	激光位姿角度。

LxLocation

字段	ctypes 类型	说明
timestamp	c_longlong	定位结果时间戳。
status	c_long	定位状态。
x	c_double	定位 x。
y	c_double	定位 y。
theta	c_double	定位角度。
extents	c_long_Array_8192	扩展字段。

LxAvState

名称	值	说明
LxAvSuccess	0	避障算法正常。
LxAvGroundPriorNotSet	-1	地面先验未设置。
LxAvInputNotSet	-2	输入未设置。
LxAvNoPointInRange	-3	范围内无点。
LxAvNoPointAfterFilterGround	-4	滤除地面后无点。
LxAvNoPointAfterRadiusFilter	-5	半径滤波后无点。
LxAvObstaclesNotSegmented	-6	障碍物未分割。
LxAvUndefinedError	-99	未定义错误。

九、API 说明**9.1 版本、日志和全局配置**

接口	返回值	说明
DcGetApiVersion()	str	获取当前实际加载的底层 API 版本。
DcSetInfoOutput(print_level,enable_screen_print,log_path,language=0)	LX_STATE	设置日志等级、是否屏幕输出、日志目录和语言。print_level 为 0/1/2，对应全部、警告、错误。
DcLog(string)	LX_STATE	向 SDK 日志输出自定义调试信息。
DcGetErrorString(state)	str	根据 LX_STATE 获取底层错误描述。参数必须是 LX_STATE 对象。

9.2 查找、连接与关闭相机

接口	返回值	说明
DcGetDeviceList()	LX_STATE,dev_list,dev_num	搜索相机列表。dev_list 是 LxDeviceInfo 指针数组，dev_num 是数量。

接口	返回值	说明
DcOpenDevice(open_mode,param)	LX_STATE,DcHandle,LxDeviceInfoInfo	按指定方式打开设备。 open_mode 必须是 LX_OPEN_MODE。
DcCloseDevice(handle)	LX_STATE	关闭设备并释放控制权限。

9.3 开始取流与关闭取流

接口	返回值	说明
DcStartStream(handle)	LX_STATE	开始数据流。启流前应完成需要在启流前设置的参数。
DcStopStream(handle)	LX_STATE	停止数据流。关闭设备前建议先停止取流。

9.4 读取与设置相机参数

接口	返回值	说明
DcSetIntValue(handle,cmd,value)	LX_STATE	设置 LX_INT 参数。value 可以是 int 或 Enum。
DcGetIntValue(handle,cmd)	LX_STATE,LxIntValueInfo	读取 LX_INT 参数及其范围信息。
DcSetFloatValue(handle,cmd,value)	LX_STATE	设置 LX_FLOAT 参数。
DcGetFloatValue(handle,cmd)	LX_STATE,LxFloatValueInfo	读取 LX_FLOAT 参数及其范围信息。
DcSetBoolValue(handle,cmd,value)	LX_STATE	设置 LX_BOOL 参数。
DcGetBoolValue(handle,cmd)	LX_STATE,bool	读取 LX_BOOL 参数。
DcSetStringValue(handle,cmd,value)	LX_STATE	设置 LX_STRING 参数，字符串会按 UTF-8 传入底层库。
DcGetStringValue(handle,cmd)	LX_STATE,str	读取 LX_STRING 参数，失败时返回空字符串。
DcSetCmd(handle,cmd)	LX_STATE	执行 LX_CMD 命令。

9.5 图像、点云和相机参数读取

接口	返回值	说明
getFrame(handle)	LX_STATE,FrameInfo	内部执行 LX_CMD_GET_NEW_FRAME, 成功后返回 FrameInfo。
getRGBImage(frame)	LX_STATE,numpy.ndarray	从 FrameInfo.rgb_data 读取 RGB 图像, shape 为 (height,width,channel)。
getDepthImage(frame)	LX_STATE,numpy.ndarray	从 FrameInfo.depth_data 读取深度图, shape 为(height,width)。
getAmpImage(frame)	LX_STATE,numpy.ndarray	从 FrameInfo.amp_data 读取强度图, shape 为(height,width)。
getPointCloud(handle)	LX_STATE,numpy.ndarray	读取 XYZ 点云, shape 为 (depth_height, depth_width,3)。调用前必须开启 3D 深度流并刷新帧。
DcSaveXYZ(handle,filename)	LX_STATE	保存点云, 支持 txt、ply、pcd。txt 按图像顺序保存全部数据, ply/pcd 通常只保存非零数据。
get2DIntricParam(handle)	LX_STATE,intrinsic,distort	读取 2D 相机内参, intrinsic 为 [fx,fy,cx,cy], distort 为 [d1,d2,d3,d4,d5]。
get3DIntricParam(handle)	LX_STATE,intrinsic,distort	读取 3D 相机内参。
get3DTransMatrix(handle)	LX_STATE,numpy.ndarray	读取 3D 外参, 返回 shape 为(3,4)的矩阵, 前 3 列为旋转矩阵, 第 4 列为平移向量。

9.6 特殊控制、ROI 和算法输出

接口	返回值	说明
DcSetCameraIp(handle,ip,netmask="gateway")	LX_STATE	设置相机 IP、子网掩码和网关。设置后相机会自动重启。
DcSpecialControl(handle,command,value=None)	LX_STATE,Any	调用扩展控制命令, 例如 GetObstacleIO、SetLaserData、ImportLocationMapFile 等。

接口	返回值	说明
DcSetRoI(handle,offset_x,offset_y,width,height,img_type)	LX_STATE	设置 ROI。img_type 为 0 表示 3D 图像，1 表示 2D 图像。当前源码中底层函数大小写需确认。
getAlgorithmStatus(handle)	LX_STATE,Any	根据 LX_INT_ALGORITHM_MODE 返回对应算法结构体。

9.7 状态回调

接口	返回值	说明
DcRegisterCameraStatusCallback(handle,function)	LX_STATE	注册相机状态回调。func 接收 CameraStatus 指针。
DcUnregisterCameraStatusCallback(handle)	LX_STATE	注销相机状态回调。

十、示例程序说明

10.1 示例程序索引

示例	主要功能	当前状态
single_camera.py	单相机同步取流、状态回调、2D/3D 流开关、图像显示、点云读取、帧率读取。	建议作为 Python 基础开发的主要参考示例。
application_obstacle.py	切换避障 V2 算法、读写算法参数、启流后读取 GetObstacleIO 和算法输出。	示例流程可参考；若使用当前 lx_camera_api.py 接口，应将设备搜索部分调整为 state,dev_list,dev_num = camera.DcGetDeviceList()的写法。
application_pallet.py	切换托盘算法、读写算法参数、读取 LxPalletPose 中的 x、y、yaw。	示例中设备搜索写法需按当前 DcGetDeviceList 返回值修正。
application_location.py	视觉定位相关结构和 SetLaserData、ImportLocationMapFile 等命令参考。	源码标注 TODO NOT FINISHED, DO NOT USE IT，正式项目使用前需补全验证。
utils.py	图像归一化显示和错误状态检查。	可复制到项目中按需要调整。

10.2 single_camera.py: 单相机同步取流

single_camera.py 是当前 Python 包中最完整的基础示例。它按 IP、ID、SN 或索引选择打开方式，注册状态回调，开启 2D、3D 深度和 3D 强度流，读取图像尺寸和数据类型，启流后循环调用 getFrame，并通过 getDepthImage、getAmpImage、getRGBImage 转换图像数据。

阶段	关键接口	说明
搜索设备	DcGetDeviceList	返回 state、dev_list 和 dev_num，dev_num<=0 时退出。
打开设备	DcOpenDevice	示例默认按 IP 打开，可切换为 SN、ID 或索引。
状态监控	DcRegisterCameraStatusCallback	测试状态回调后再注销。实际项目可保持注册直到关闭设备前。
启用数据流	DcSetBoolValue	分别控制 LX_BOOL_ENABLE_2D_STREAM、 LX_BOOL_ENABLE_3D_DEPTH_STREAM、 LX_BOOL_ENABLE_3D_AMP_STREAM。
读取图像信息	DcGetIntValue	读取宽高、通道数和数据类型。
刷新帧	getFrame	循环中刷新 FrameInfo。
图像转换	getDepthImage/getAmpImage/getRGBImage	返回 numpy.ndarray，可使用 OpenCV 显示。
点云	getPointCloud	读取 XYZ 点云；示例中使用 open3d 可可视化。

10.3 application_obstacle.py: 避障算法

避障示例将 LX_INT_ALGORITHM_MODE 设置为 MODE_AVOID_OBSTACLE2，读取算法版本和参数 JSON，必要时修改参数后写回。启流后循环 getFrame，并通过 DcSpecialControl(handle,'GetObstacleIO')读取 IO 结果，通过 getAlgorithmStatus 读取 LxAvoidanceOutputN。

10.4 application_pallet.py: 托盘对接算法

托盘示例将 LX_INT_ALGORITHM_MODE 设置为 MODE_PALLET_LOCATE，读取算法参数后进入取流循环，并通过 getAlgorithmStatus 获取 LxPalletPose。常用字段为 x、y 和 yaw，分别表示托盘位姿输出。

10.5 application_location.py: 视觉定位参考

视觉定位示例当前未完成，源码中已明确标注不建议直接使用。该文件仍可用于了解 LxLaser、SetLaserData、ImportLocationMapFile 等结构和控制命令的调用方向。正式项目中，应先确认算法模式、地图文件格式、激光数据生命周期和 DcSpecialControl 参数传递方式。另外，当前 application_location.py 中设置的算法模式并不是 MODE_VISION_LOCATION，而是 MODE_AVOID_OBSTACLE2，因此该文件只能作为结构体和 DcSpecialControl 调用方式参考，不能作为完整视觉定位示例直接使用。

10.6 utils.py: 错误处理与图像显示

utils.py 中的 visualize 会把图像归一化到 0 到 255 并应用 OpenCV 伪彩色，适合调试深度图和强度图。check_state 会在非 LX_SUCCESS 时输出错误字符串，并在传入 handle 时关闭设备。项目中可按自身异常处理方式改成 raise 异常或日志记录。

十一、调试与常见问题

表 11-1 常见问题排查

问题	最可能原因	处理方式
LxCamera 初始化失败	dll_path 路径错误、动态库后缀与平台不匹配，或 Python 架构与 DLL 架构不一致。	确认文件存在；Windows 使用 .dll，Linux 使用 .so；64 位 Python 加载 win_x64 动态库。
搜索不到相机	主机和相机不在同一网段、防火墙拦截、网线或供电异常。	检查 192.168.100.* 网段、关闭或放行防火墙、确认网口协商速率和供电。
DcOpenDevice 失败	open_mode 和 param 不匹配，IP/SN/ID 填写错误，或相机控制权限被占用。	用 DcGetDeviceList 确认设备信息；关闭其他占用相机的程序；等待心跳超时后重试。
getFrame 返回非成功	未启流、网络异常、帧同步超时或多机干扰。	确认 DcStartStream 成功；对 LX_E_RECONNECTING 等待重连；对 LX_E_FRAME_ID_NOT_MATCH 和 LX_E_FRAME_MULTI_MACHINE 按现场同步和多机模式处理。
图像指针为空	对应数据流未开启，或启流后还没有有效帧。	先设置对应 LX_BOOL_ENABLE_*_STREAM 为 True，再启流并调用 getFrame。

问题	最可能原因	处理方式
getPointCloud 失败	未开启 3D 深度流，或未刷新新帧。	先开启 LX_BOOL_ENABLE_3D_DEPTH_STREAM，并调用 getFrame 或 DcSetCmd(LX_CMD_GET_NEW_FRAME)。
算法参数读取失败	未先设置 LX_INT_ALGORITHM_MODE，或当前设备不支持该算法。	先设置算法模式，再读取 LX_STRING_ALGORITHM_VERSION 和 LX_STRING_ALGORITHM_PARAMETERS。
DcSetRoI 不可用	Python 封装与底层符号名大小写不一致，或设备/当前状态不允许设置 ROI。	确认动态库导出名；必要时修正封装；停流后再设置 ROI，并重新读取图像宽高。
OpenCV 窗口不显示	opencv-python 未安装，或运行环境不支持 GUI 显示。	安装 opencv-python；无显示环境时只保存图像或打印 shape。

十二、版本与兼容说明

表 12-1 版本与兼容注意事项

项目	说明
Python 包版本	当前 whl 为 lx-camera-py 1.3.3，源码包 dist-info 中声明依赖 numpy 和 opencv-python。
封装版本检查	lx_camera_api.py 中 SDK_VERSION 为 2.4.30。初始化 LxCamera 时会调用 DcGetApiVersion，如果加载版本小于或等于该值，会写日志并打印版本不匹配提示。
底层 SDK 版本	当前 SDK/include/lx_camera_api_version.h 中版本宏为 LX_CAMERA_API_VERSION V2.4.60,20260126。实际运行以 LxCamera 加载的动态库版本为准。
平台支持	Python 封装按 sys.platform 判断 Windows 和 Linux，其他平台会抛出 NotImplementedError。
接口覆盖范围	当前 Python 封装未覆盖 C/C++ 指南中的全部接口，例如帧回调、IMU 回调、GPU/PTP/JPEG 解码配置等未在 LxCamera 类中封装。需要这些能力时，应扩展 ctypes 封装或使用 C/C++ 接口。
示例状态	single_camera.py 较完整；application_location.py 仍为未完成示例；部分算法示例需按当前 DcGetDeviceList 返回值修正后再运行。
需确认事项	ROI 底层函数命名、应用算法 DcSpecialControl 命令参数、视觉定位地图文件格式和激光数据传递方式，应在具体设备和算法版本上验证。